Módulo 2: React

Sumário

Sumário..... 2
1. Introdução ao React – O Poder da Interface Baseada em Componentes..... 3
1.1. O que é o React e por que ele surgiu?..... 3
1.2. O Conceito Fundamental: Componentes..... 3
1.3. O Segredo do JSX (JavaScript XML)..... 4
2. Props (Propriedades) – Tornando os Componentes Dinâmicos e Reutilizáveis..... 4
2.1. O que são as Props?..... 5
2.2. Passando e Recebendo Props na Prática..... 5
2.3. Desestruturação de Props (Destructuring)..... 6
3. Estado (useState) – Dando Memória e Dinamismo aos Componentes..... 7
3.1. O que é o Estado?..... 7
3.2. O Hook useState..... 8
3.3. Exemplo Prático: Um Contador Interativo..... 8
3.4. O Estado é Assíncrono..... 10
4. Renderização Condicional – Controlando o Que Aparece na Tela..... 10
4.1. Renderização Condicional: Mostrando ou Escondendo Elementos..... 10
4.2. Renderização de Listas: O Método .map()..... 12
4.3. A Importância Crítica da Propriedade key..... 12
5. Manipulação de Formulários – Capturando e Controlando a Entrada do Usuário..... 13
5.1. O Conceito de Inputs Controlados..... 13
5.2. Construindo um Formulário Controlado na Prática..... 14
5.3. Lendo Múltiplos Inputs com um Único Estado (Avançado)..... 16
6. Comunicação Assíncrona e Efeitos Colaterais – Conectando o React ao Mundo Externo..... 17
6.1. O Conceito de Efeitos Colaterais (Side Effects) e o useEffect..... 18
6.2. Buscando Dados de uma API (Fetch API ou Axios)..... 19
6.3. O Ciclo de Vida da Requisição Assíncrona..... 20

1. Introdução ao React – O Poder da Interface Baseada em Componentes

Seja bem-vindo ao módulo principal sobre React! Se nos módulos anteriores focamos em persistência de dados, CRUD e backend com Node.js, agora vamos cruzar a fronteira para o desenvolvimento frontend moderno.

Neste primeiro capítulo, vamos desvendar o que é o React, por que ele revolucionou a criação de interfaces web e qual é o conceito central que sustenta toda a sua arquitetura: os componentes.

1.1. O que é o React e por que ele surgiu?

Criado e mantido pelo Facebook (Meta), o React é uma biblioteca JavaScript declarativa e eficiente para a construção de interfaces de usuário (User Interfaces ou UIs).

Antigamente, o desenvolvimento web tradicional dependia de manipulação direta do DOM (Document Object Model) — o que significava escrever códigos imperativos dizendo ao navegador exatamente o que alterar, esconder ou adicionar passo a passo (document.getElementById().innerHTML = ...). Em aplicações grandes, esse modelo tornava o código confuso, difícil de manter e propenso a bugs de sincronização de estado.

O React mudou essa lógica ao introduzir uma abordagem declarativa:

- Você descreve como a interface deve se parecer com base em um determinado estado de dados.

- Quando os dados mudam, o próprio React calcula de forma inteligente e eficiente o que precisa ser atualizado na tela, alterando apenas os elementos necessários através do seu Virtual DOM.

1.2. O Conceito Fundamental: Componentes

Se você olhar para qualquer site moderno (como o YouTube, o GitHub ou um painel de controle), perceberá que a tela é composta por repetições de elementos visuais: barras de navegação, cards de produtos, botões, avatares e listas.

O React propõe que você divida a sua interface inteira em pequenos blocos independentes, reutilizáveis e isolados chamados Componentes.

- O que é um componente? Em termos práticos, um componente React é uma função em JavaScript que retorna um pedaço de estrutura visual (parecido com HTML).

- Reutilização: Um componente de "Botão" ou "Card de Usuário" pode ser escrito uma única vez e reutilizado dezenas de vezes em diferentes páginas da aplicação, recebendo dados diferentes através de propriedades.

1.3. O Segredo do JSX (JavaScript XML)

Quando você olha para o código de um componente React, vai se deparar com algo curioso: código HTML misturado diretamente dentro do arquivo JavaScript.

```
// Exemplo de um componente React básico
function Saudacao() {
    return (
        <div>
            <h1>Olá, Gabriel!</h1>
            <p>Seja bem-vindo ao módulo de React.</p>
        </div>
    );
}
```

Essa sintaxe chama-se JSX (JavaScript XML).

- Ela permite escrever marcações visuais diretamente no código JavaScript de forma fluida e intuitiva.

- Por baixo dos panos, o compilador (como o Vite ou o Babel) transforma esse JSX em chamadas puras de funções JavaScript antes de o navegador exibi-lo na tela.

2. Props (Propriedades) – Tornando os Componentes Dinâmicos e Reutilizáveis

No Capítulo 1, aprendemos que o React divide a interface em pequenos blocos independentes chamados Componentes. No entanto, se um componente fosse totalmente estático (exibindo sempre o mesmo texto e as mesmas imagens), ele não teria utilidade real em uma aplicação dinâmica.

É aqui que entram as Props (abreviação de Properties ou Propriedades). Elas são o mecanismo fundamental para passar dados de um componente pai para um componente filho.

2.1. O que são as Props?

Pense nas props de um componente exatamente como os parâmetros de uma função em JavaScript ou os argumentos de uma tag HTML ().

- Enquanto o HTML padrão possui atributos fixos (src, href, className), os componentes React permitem que você crie seus próprios atributos personalizados.

- As props são imutáveis do ponto de vista do componente que as recebe (ou seja, o componente filho pode ler e exibir os dados, mas nunca modificá-los diretamente; quem controla o dado é sempre o componente pai).

2.2. Passando e Recebendo Props na Prática

Imagine que estamos desenvolvendo um sistema de portfólio ou um painel de usuários e queremos criar um componente de "Card de Perfil" reutilizável.

Em vez de criar um card rígido para cada pessoa, criamos um componente genérico e passamos as informações dinamicamente via props.

A. Criando o Componente Filho (CardPerfil.jsx)

O componente recebe um argumento (por convenção chamado de props) e utiliza as chaves {} para injetar esses valores no JSX:

```
// CardPerfil.jsx
function CardPerfil(props) {
    return (
        <div className="card">
            <h2>{props.nome}</h2>
            <p>Cargo: {props.cargo}</p>
            <p>Cidade: {props.cidade}</p>
        </div>
    );
}

export default CardPerfil;
```

B. Renderizando e Passando os Dados no Componente Pai (App.jsx)

Agora, chamamos o nosso componente várias vezes a partir de um componente pai, injetando valores diferentes em cada chamada através das props:

```
// App.jsx
import CardPerfil from './CardPerfil';

function App() {
    return (
        <div>
            <h1>Equipe de Desenvolvimento</h1>

            {/* Passando props personalizadas para o componente
*/}
            <CardPerfil
                nome="Gabriel Marques"
                cargo="Engenheiro de QA & Backend"
                cidade="Curitiba"
            />

            <CardPerfil
                nome="Ana Souza"
                cargo="Desenvolvedora Frontend"
                cidade="São Paulo"
            />
        </div>
    );
}

export default App;
```

2.3. Desestruturação de Props (Destructuring)

À medida que os componentes crescem, escrever props.nome, props.cargo, props.cidade repetidas vezes pode se tornar cansativo.

Uma excelente prática em JavaScript moderno (e muito utilizada no ecossistema React) é a desestruturação (destructuring), onde extraímos as propriedades diretamente nos parâmetros da função do componente:

```jsx
// Usando desestruturação para deixar o código mais limpo
function CardPerfil({ nome, cargo, cidade }) {
    return (
        <div className="card">
            <h2>{nome}</h2>
            <p>Cargo: {cargo}</p>
            <p>Cidade: {cidade}</p>
        </div>
    );
}

export default CardPerfil;
```

O resultado visual é exatamente o mesmo, mas a leitura do código fica muito mais elegante e direta.

3. Estado (useState) – Dando Memória e Dinamismo aos Componentes

Nos capítulos anteriores, aprendemos que os componentes são os blocos de construção do React (Capítulo 1) e que podemos torná-los reutilizáveis passando dados de pai para filho através das Props (Capítulo 2).

No entanto, as props são fixas e controladas por quem está de fora (o componente pai). O que acontece quando o usuário interage com a tela — clica em um botão, digita em um campo de formulário, ativa um tema escuro ou abre um menu — e o componente precisa se lembrar dessa mudança e atualizar a interface?

É exatamente aqui que entra o conceito mais importante do React: o Estado (State).

3.1. O que é o Estado?

Se as props funcionam como os argumentos passados para uma função, o Estado funciona como a memória interna do componente.

- Variáveis comuns não funcionam: Em JavaScript puro, se você alterar o valor de uma variável comum (let contador = 0; contador++), o valor muda na memória, mas o navegador não redesenha a tela para mostrar o novo número. O React não "vê" essa alteração acontecer.

- O poder do Estado: O Estado é uma variável especial monitorada pelo React. Sempre que o valor de um estado é alterado, o React automaticamente re-renderiza o componente (atualiza a interface em tempo real) para refletir o novo dado na tela.

3.2. O Hook useState

Para criar e gerenciar um estado em um componente funcional, utilizamos uma função especial fornecida pelo próprio React chamada useState (conhecida no ecossistema como um Hook).

A sintaxe fundamental:

```
import { useState } from 'react';

function MeuComponente() {
  // [valorAtual, funcaoParaAtualizarOValor] =
useState(valorInicial);
  const [contador, setContador] = useState(0);
}
```

Analisando a estrutura:

- useState(0): Define que o valor inicial do nosso estado contador será 0.
- O useState retorna um array com duas posições:
    - O valor atual (contador): Armazena o dado no momento presente.
    - A função modificadora (setContador): A única forma recomendada de alterar o valor do estado. Quando você a chama, o React atualiza o dado e redesenha a tela.

3.3. Exemplo Prático: Um Contador Interativo

Vamos construir um componente simples que possui um número na tela e dois botões: um para incrementar e outro para decrementar.

```
import { useState } from 'react';

function Contador() {
  // Inicializamos o estado 'contador' com o valor 0
  const [contador, setContador] = useState(0);

  function incrementar() {
    // Atualizamos o estado chamando a função setContador
    setContador(contador + 1);
  }
```

```
  function decrementar() {
    setContador(contador - 1);
  }

  return (
    <div style={{ padding: '20px', fontFamily: 'sans-serif' }}>
      <h2>Contador Interativo</h2>
      <p style={{ fontSize: '24px', fontWeight: 'bold' }}>Valor
atual: {contador}</p>

      {/* Vinculamos os eventos de clique às nossas funções */}
      <button onClick={incrementar} style={{ marginRight: '10px',
padding: '8px 16px' }}>
        Incrementar (+)
      </button>

      <button onClick={decrementar} style={{ padding: '8px 16px'
}}>
        Decrementar (-)
      </button>
    </div>
  );
}

export default Contador;
```

O que acontece por trás dos panos quando o botão é clicado?

- O usuário clica em "Incrementar (+)".
- A função incrementar é executada e chama setContador(contador + 1).
- O React percebe que o estado mudou.
- Ele roda o componente Contador novamente, atualizando a variável {contador} para o novo valor.
- O navegador exibe a nova contagem instantaneamente na tela.

3.4. O Estado é Assíncrono

Uma pegadinha clássica para quem está começando: as atualizações de estado no React não acontecem de forma síncrona imediata na mesma linha de código. O React agrupa as alterações de estado para otimizar a performance (processo conhecido como batching).

Portanto, se você precisar atualizar um estado baseando-se no valor anterior dele mesmo, a prática recomendada é passar uma função de callback para o modificador:

```
// Maneira mais segura de atualizar estados baseados no valor
anterior:
setContador(estadoAnterior => estadoAnterior + 1);
```

4. Renderização Condicional e de Listas – Controlando o Que Aparece na Tela

Nos capítulos anteriores, vimos como estruturar componentes (Capítulo 1), como passar dados de pai para filho usando Props (Capítulo 2) e como dar memória e reatividade aos componentes usando o Estado com o useState (Capítulo 3).

Agora, vamos dar o próximo passo lógico: Como controlar a exibição dos elementos na tela com base em condições lógicas e como transformar arrays de dados em listas visuais dinâmicas?

4.1. Renderização Condicional: Mostrando ou Escondendo Elementos

Em uma aplicação real, precisamos exibir diferentes conteúdos dependendo da situação:

- Mostrar um botão de "Sair" apenas se o usuário estiver logado.
- Exibir um aviso de "Carregando..." enquanto os dados de uma API não chegam.
- Mostrar uma mensagem de erro se um formulário estiver preenchido incorretamente.

O React utiliza a própria lógica do JavaScript (if, operador ternário ? : e o operador lógico &&) para decidir o que renderizar.

A. O Operador Ternário (condicao ? verdadeiro : falso)

É o formato mais utilizado no dia a dia do React para alternar entre dois blocos visuais de acordo com o estado.

```
import { useState } from 'react';

function StatusLogin() {
  const [estaLogado, setEstaLogado] = useState(false);

  return (
    <div style={{ padding: '20px' }}>
      {/* Usamos o operador ternário dentro do JSX */}
      {estaLogado ? (
        <h2>Bem-vindo de volta, usuário! </h2>
      ) : (
        <h2>Por favor, faça login para continuar. </h2>
      )}

      <button onClick={() => setEstaLogado(!estaLogado)}>
        {estaLogado ? 'Sair' : 'Entrar'}
      </button>
    </div>
  );
}

export default StatusLogin;
```

B. O Operador Lógico && (E Curto-Circuito)

Quando você quer exibir um elemento apenas se uma condição for verdadeira (e não quer renderizar nada caso seja falsa), o operador && é a escolha perfeita.

```
function Notificacao({ temMensagens }) {
  return (
    <div>
      <h3>Painel Principal</h3>

      {/* Se temMensagens for true, o alerta é exibido. Se for
false, o React ignora. */}
      {temMensagens && <p style={{ color: 'red' }}>Você tem novas
mensagens não lidas! </p>}
    </div>
  );
}
```

4.2. Renderização de Listas: O Método .map()

Muitas vezes, não temos apenas um item fixo para exibir, mas sim uma lista de dados vinda de um banco de dados ou API (como uma lista de produtos de um e-commerce, tarefas de uma lista de afazeres ou um ranking de pontuação).

Em JavaScript, transformamos arrays utilizando o método .map(). No React, fazemos exatamente a mesma coisa para transformar um array de dados em um array de componentes JSX.

Exemplo prático: Renderizando uma lista de tarefas

```
function ListaTarefas() {
  const tarefas = [
    { id: 1, texto: 'Estudar fundamentos do React' },
    { id: 2, texto: 'Praticar hooks (useState)' },
    { id: 3, texto: 'Criar aplicação frontend' }
  ];

  return (
    <div style={{ padding: '20px' }}>
      <h2>Minha Lista de Tarefas</h2>
      <ul>
        {/* O .map percorre o array e retorna um elemento <li>
para cada item */}
        {tarefas.map((tarefa) => (
          <li key={tarefa.id}>{tarefa.texto}</li>
        ))}
      </ul>
    </div>
  );
}

export default ListaTarefas;
```

4.3. A Importância Crítica da Propriedade key

Se você reparar no exemplo acima, a tag <li> possui um atributo chamado key:

```
<li key={tarefa.id}>{tarefa.texto}</li>
```

- O que é a key? É uma identificação única que o React exige sempre que você renderiza uma lista de elementos.

- Por que ela é obrigatória? Quando um item da lista é adicionado, removido

ou reordenado, o React usa a key para rastrear exatamente qual elemento mudou, atualizando o Virtual DOM de forma extremamente eficiente sem precisar recriar a lista inteira do zero.

- Boa prática: Sempre utilize um identificador único vindo dos seus dados (como id_usuario, id_produto). Evite usar o índice do array (index) como chave, pois isso pode causar bugs visuais graves se a ordem da lista mudar.

5. Manipulação de Formulários – Capturando e Controlando a Entrada do Usuário

Nos capítulos anteriores, aprendemos a estruturar componentes (Capítulo 1), passar dados via Props (Capítulo 2), gerenciar a memória com o Estado usando o useState (Capítulo 3) e controlar o que aparece na tela através de renderização condicional e listas (Capítulo 4).

Agora, vamos dar um passo fundamental para qualquer aplicação interativa: como coletar dados digitados pelos usuários através de formulários e inputs.

5.1. O Conceito de Inputs Controlados

No HTML tradicional, os elementos de formulário (como <input>, <select>, <textarea>) mantêm o seu próprio estado internamente no navegador. Se você digita algo em um input, o próprio navegador gerencia aquele texto.

- No React, mudamos essa lógica para garantir o controle total dos dados. Adotamos o padrão de Inputs Controlados:

- O valor exibido no input é ditado estritamente por uma variável de Estado (useState).

- Sempre que o usuário digita algo, dispararmos uma função que atualiza esse estado imediatamente.

- Dessa forma, o componente React é a "única fonte da verdade" para o valor daquele campo.

5.2. Construindo um Formulário Controlado na Prática

Vamos criar um componente de cadastro simples onde o usuário digita seu nome e e-mail, e esses dados são capturados e exibidos quando o formulário é enviado.

```
import { useState } from 'react';

function FormularioCadastro() {
  // Criamos estados separados para cada campo do formulário
  const [nome, setNome] = useState('');
  const [email, setEmail] = useState('');

  // Função disparada quando o formulário é submetido
  function handleSubmit(evento) {
    // Previne o comportamento padrão do navegador de recarregar a
página inteira
    evento.preventDefault();

    console.log('Dados submetidos:', { nome, email });
    alert(`Usuário cadastrado: nome({email})`);

    // Opcional: limpar os campos após o envio
    setNome('');
    setEmail('');
  }

  return (
    <form onSubmit={handleSubmit} style={{ padding: '20px',
maxWidth: '300px' }}>
      <h2>Cadastro de Usuário</h2>

      <div style={{ marginBottom: '15px' }}>
        <label>Nome:</label>
        <br />
        <input
          type="text"
          value={nome} // O valor do input é amarrado ao estado
          onChange={(e) => setNome(e.target.value)} // Atualiza o
estado a cada tecla digitada
          placeholder="Digite seu nome"
          style={{ width: '100%', padding: '8px', boxSizing:
'border-box' }}
        />
```

```
      </div>

      <div style={{ marginBottom: '15px' }}>
        <label>E-mail:</label>
        <br />
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="Digite seu e-mail"
          style={{ width: '100%', padding: '8px', boxSizing:
'border-box' }}
        />
      </div>

      <button type="submit" style={{ padding: '10px 15px', cursor:
'pointer' }}>
        Cadastrar
      </button>
    </form>
  );
}

export default FormularioCadastro;
```

Detalhes cruciais deste código:

- value={nome}: Garante que o input reflita exatamente o que está guardado no estado.

- onChange={(e) => setNome(e.target.value)}: O evento onChange captura cada alteração. e.target.value extrai o texto que acabou de ser digitado e atualiza o estado instantaneamente.

- evento.preventDefault(): No evento de envio (onSubmit), essa linha impede que o navegador recarregue a página (comportamento nativo do HTML), permitindo que o React processe os dados de forma assíncrona (ótimo para enviar depois via API para o nosso backend em Node.js!).

5.3. Lendo Múltiplos Inputs com um Único Estado (Avançado)

Se o seu formulário tiver 10 ou 15 campos diferentes, criar um useState para cada um deles (nome, email, telefone, endereco, etc.) pode deixar o código verboso.

Uma prática muito comum e elegante é agrupar todos os campos em um único objeto de estado:

```
import { useState } from 'react';

function FormularioCompleto() {
  // O estado agora é um objeto contendo todas as propriedades do formulário
  const [formulario, setFormulario] = useState({
    nome: '',
    email: '',
    cidade: ''
  });

  function handleChange(evento) {
    const { name, value } = evento.target;

    // Atualizamos apenas a propriedade correspondente usando a sintaxe de espalhamento (...)
    setFormulario(estadoAnterior => ({
      ...estadoAnterior,
      [name]: value
    }));
  }

  function handleSubmit(e) {
    e.preventDefault();
    console.log('Formulário enviado:', formulario);
  }

  return (
    <form onSubmit={handleSubmit} style={{ padding: '20px' }}>
      <input
        type="text"
        name="nome"
        value={formulario.nome}
        onChange={handleChange}
        placeholder="Nome"
```

```
      />
      <br /><br />
      <input
        type="email"
        name="email"
        value={formulario.email}
        onChange={handleChange}
        placeholder="E-mail"
      />
      <br /><br />
      <input
        type="text"
        name="cidade"
        value={formulario.cidade}
        onChange={handleChange}
        placeholder="Cidade"
      />
      <br /><br />
      <button type="submit">Enviar</button>
    </form>
  );
}

export default FormularioCompleto;
```

- O operador de espalhamento (...estadoAnterior): Garante que, ao alterarmos o campo nome, os campos email e cidade não sejam apagados, preservando o restante do objeto de estado.

- A propriedade name: Serve como chave dinâmica ([name]: value) para identificar qual campo específico está sendo alterado na função handleChange.

6. Comunicação Assíncrona e Efeitos Colaterais – Conectando o React ao Mundo Externo

Nos capítulos anteriores, aprendemos a estruturar componentes (Capítulo 1), passar dados via Props (Capítulo 2), gerenciar o estado local com o useState (Capítulo 3), controlar a renderização e listas (Capítulo 4) e capturar entradas de usuários através de formulários (Capítulo 5).

Até agora, todo o funcionamento das nossas aplicações React era puramente interno (estava na memória do navegador). Mas como um frontend exibe produtos vindos de um banco de dados, lista usuários cadastrados ou envia um formulário para uma API Node.js?

É exatamente isso que vamos resolver neste capítulo explorando a comunicação assíncrona e o Hook useEffect.

6.1. O Conceito de Efeitos Colaterais (Side Effects) e o useEffect

Em um componente React, a função principal de renderização deve ser "pura": ela recebe props e estado e retorna JSX. No entanto, tarefas como buscar dados de uma API, alterar o título da aba do navegador, interagir com o localStorage ou configurar um timer não fazem parte estritamente da renderização visual. Elas são chamadas de Efeitos Colaterais (Side Effects).

Para executar esses efeitos no momento correto do ciclo de vida de um componente, utilizamos o Hook useEffect.

A estrutura básica do useEffect:

```
import { useEffect, useState } from 'react';

function MeuComponente() {
  useEffect(() => {
    // Código do efeito colateral é executado aqui
    console.log('O componente apareceu na tela!');
  }, []); // Array de dependências
}
```

O segredo do Array de Dependências ([]):

O segundo argumento do useEffect é um array que controla quando o efeito deve ser executado:

- Array vazio ([]): O código de dentro do efeito roda apenas uma vez, exatamente quando o componente é montado (aparece pela primeira vez na tela). É o local perfeito para buscar dados iniciais de uma API.

- Array com variáveis ([idUsuario]): O código roda na montagem e sempre que a variável dentro do array mudar de valor.

- Sem array: O efeito roda a cada ciclo de re-renderização do componente (geralmente evitado para prevenir loops infinitos).

6.2. Buscando Dados de uma API (Fetch API ou Axios)

Para se comunicar com um servidor externo ou com a nossa própria API em Node.js, utilizamos funções assíncronas do JavaScript (async/await) combinadas com o método nativo fetch (ou bibliotecas como o axios).

Vamos construir um componente que busca uma lista de usuários de uma API pública e os exibe na tela:

```
import { useState, useEffect } from 'react';

function ListaUsuariosAPI() {
  const [usuarios, setUsuarios] = useState([]);
  const [carregando, setCarregando] = useState(true);

  // useEffect para buscar os dados assim que o componente
carregar
  useEffect(() => {
    async function buscarDados() {
      try {
        const resposta = await
fetch('https://jsonplaceholder.typicode.com/users');
        const dados = resposta.json(); // Converte o retorno para
JSON

        setUsuarios(dados); // Atualiza o estado com os dados da
API
      } catch (erro) {
        console.error('Erro ao buscar usuários:', erro);
      } finally {
        setCarregando(false); // Desativa o indicador de
carregamento independentemente do sucesso ou erro
      }
    }

    buscarDados();
  }, []); // Array vazio garante que a requisição ocorra apenas
uma vez na montagem

  return (
    <div style={{ padding: '20px' }}>
      <h2>Usuários Buscados da API</h2>

      {/* Renderização condicional para o estado de carregamento
*/}
```

```
      {carregando ? (
        <p>Carregando dados do servidor... ⏳</p>
      ) : (
        <ul>
          {usuarios.map((usuario) => (
            <li key={usuario.id}>
              <strong>{usuario.name}</strong> ({usuario.email})
            </li>
          ))}
        </ul>
      )}
    </div>
  );
}

export default ListaUsuariosAPI;
```

6.3. O Ciclo de Vida da Requisição Assíncrona

Quando criamos aplicações que conversam com o backend, lidamos com três estados fundamentais na interface:

- O Estado Inicial (Carregamento / Loading): O usuário abre a página, o useEffect dispara a requisição e a tela exibe um aviso de carregamento (carregando = true).

- O Sucesso (Dados Prontos): A API responde com os dados, o estado é atualizado (setUsuarios(dados)), o carregamento é encerrado (setCarregando(false)), e o React redesenha a lista na tela.

- A Falha (Tratamento de Erro): Se a rede cair ou o servidor falhar, o bloco catch captura o erro e impede que a aplicação quebre, permitindo exibir uma mensagem amigável ao usuário.